CAPSTONE PROJECT WORKBOOK · INTERNAL TRAINING USE

Python Core Refresher Capstone Project Workbook

Two industry-style projects — problem statements, PRDs, architecture & coding standards, written the way a junior engineer actually receives them.

Projects included	1. Telecom CDR & Billing Insights CLI 2. Retail Inventory & Sales Reconciliation CLI
Skills exercised	Setup, syntax, variables & data types, control flow & loops · Data structures (list/dict/set), functions & basic OOP, string manipulation · File handling (CSV/JSON), scripting best practices
Solutions included	No — this is a workbook, not a lesson. Build against the acceptance criteria.
Format	Each project: PRD → Architecture → Sample Data → 3 Milestones of numbered problem-statement questions

How to Use This Workbook

1. Read the PRD and Architecture for a project fully before writing any code — in a real job, skipping this step is how people build the wrong thing correctly.
2. Set up the folder structure described in the Architecture section first. Don't write everything into one file, even though it's tempting for something this size.
3. Work through the milestones in order — each one deliberately depends on functions/classes built in the previous milestone.
4. Test against the provided sample data before considering a milestone done.
5. Before submitting, run through the Deliverables Checklist and self-score against the Evaluation Rubric at the end of this workbook.

Contents

Deliverables Checklist & Evaluation Rubric

Engineering Coding Standards (Applies to Both Projects)

Every project in this workbook is graded against these standards, not just "does it run":

#	Standard	What It Means in Practice
1	Style	PEP8. `snake_case` for functions/variables, `PascalCase` for classes, `UPPER_CASE` for constants.
2	Docstrings	Every function/class gets a docstring: one-line summary, then `Args:` / `Returns:` if non-obvious.
3	No bare `except:`	Catch specific exception types. Not knowing what could go wrong is a sign you haven't thought through the failure mode.
4	No `print()` for status/errors	Use `logging`. `print()` is reserved for the final, intentional console summary.
5	No top-level side effects	Nothing runs just from importing a file. Guard entry points with `if __name__ == "__main__":`.
6	Constants live in one place	A `config.py` or a clearly-named constants block — never scattered magic numbers.
7	Modular structure	Follow the Architecture section's module breakdown. A single monolithic script is a rewrite, not a passing submission.
8	Type hints	Expected as documentation on function signatures, e.g. `def compute_cost(call_type: str, duration_sec: int) -> float:`

Project 1 — Telecom CDR & Billing Insights CLI

> **Client:** Orbit Mobile Pvt Ltd (fictitious regional telecom operator) · **Requested by:** Billing Operations, via the Engineering Lead

1.1 Problem Statement / PRD

Background. Orbit Mobile's billing ops team currently reconciles each day's call records by hand in a spreadsheet — pulling a raw call log, cross-referencing subscriber plans, calculating charges, and flagging anything unusual. As call volume has grown, this has become slow and error-prone. Engineering has scoped a small internal CLI tool to automate this for a single day's batch. You are the engineer assigned to build it.

Goals

- Automate ingestion and validation of a day's CDR (Call Detail Record) batch.
- Compute per-subscriber billing based on call type and duration.
- Flag suspicious call patterns for manual fraud review.
- Produce a daily summary report, both machine-readable (JSON) and human-readable (console).

Out of Scope (Phase 2, not this project)

- No live network/database integration — file-based batch processing only.
- No multi-day historical trend analysis.
- No user interface — this is a CLI tool run by ops/engineering, not customer-facing.

Stakeholders

- **Billing Ops Analyst** — consumes the daily report, needs it accurate and easy to read.
- **Engineering Lead** — reviews code quality and maintainability before this gets scheduled as a cron job.

Functional Requirements

ID	Requirement
FR1	The tool shall load a subscriber master list from a JSON file.
FR2	The tool shall load a batch of CDRs from a CSV file.

FR3	The tool shall validate each CDR row and skip (logging + counting) malformed rows without crashing the batch.
FR4	The tool shall compute a per-call cost based on call type (`domestic` / `roaming` / `international`) and duration.
FR5	The tool shall flag a call as fraud-suspect if: (a) duration > 0 but computed cost == 0, or (b) it's `international` and duration exceeds a configurable threshold (default 3600s).
FR6	The tool shall aggregate total cost and call count per subscriber.
FR7	The tool shall identify CDRs referencing an `msisdn` not present in the subscriber master list, reported separately as "unknown subscriber" anomalies.
FR8	The tool shall write a daily report as indented JSON, and print a human-readable console summary.
FR9	Input file paths and the fraud duration threshold shall be configurable via CLI arguments, not hardcoded.

Non-Functional Requirements

ID	Requirement
NFR1	All status/error events must be logged with timestamps and severity levels — no bare `print` for this.
NFR2	If more than 10% of CDR rows in a batch are malformed, the tool must log `CRITICAL` and exit with a non-zero exit code instead of writing a partial report.
NFR3	Code must be organized into separate modules (parsing/rating/fraud/reporting/IO), not one script.
NFR4	All monetary values must be rounded to 2 decimal places.

Acceptance criteria. Running the tool against the provided sample `subscribers.json` and `cdrs.csv` produces a `report.json` whose per-subscriber totals you can verify by hand-calculating at least two subscribers' costs yourself.

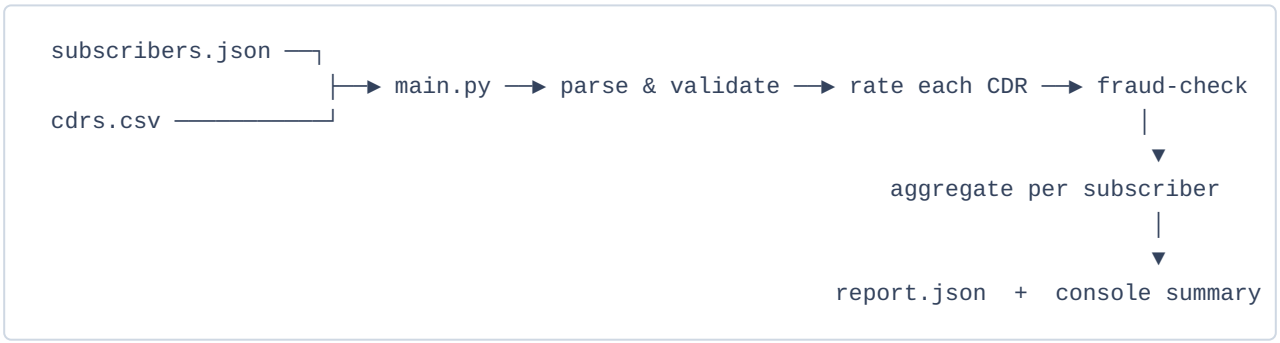
1.2 Architecture (Simple)

Module layout:

```
telecom_cli/
    main.py          # CLI entry (argparse), orchestrates the pipeline
    config.py          # RATES dict + thresholds -- the ONLY magic numbers
    io_utils.py           # load_subscribers(), load_cdrs(), write_report()
    models.py                # Subscriber class, (optionally) CDR class
    rating.py                   # compute_cost()
```

```
    fraud.py                    # is_suspicious()
    reporting.py                   # build_report()
```

Data flow:



Layered view (same shape you'll see again in the FastAPI capstone project later in the curriculum):

Layer	Responsibility	Modules
CLI Layer	Parse arguments, wire everything together, set exit code	`main.py`
Business Logic	Rating, fraud rules, aggregation	`rating.py`, `fraud.py`, `reporting.py`
Data Access	Reading/writing files	`io_utils.py`

1.3 Sample Data

Provided alongside this document in `python_core_capstone_workbook.ipynb` — running the sample-data cell generates `subscribers.json` and `cdrs.csv`, deliberately including an unknown-subscriber row and a malformed row so there's something real to validate against.

Milestone 1 — Setup, Syntax, Variables & Data Types, Control Flow & Loops

> **Q1.1** — Design the folder/module layout from section 1.2. Write a startup banner print using an f-string that includes today's date, guarded so it only runs when the script is executed directly.

> **Q1.2** — Write `parse_cdr_line(row: dict) -> dict` that returns a cleaned dict with `duration_sec` cast to `int`. Raise `ValueError` with a clear message on bad/missing data.

> **Q1.3** — Write `compute_cost(call_type: str, duration_sec: int) -> float`. For this milestone, implement the rate table (domestic 1.5, roaming 8.0, international 12.0 per minute) using `if/elif/else` — you'll refactor to a dict lookup in Milestone 2.

Q1.4 — Using a `for` loop (no comprehensions yet), iterate over a hardcoded list of at least 4 sample CDR dicts and print each one's computed cost.

Q1.5 TRICKY — Using `while`, write a retry loop simulating re-attempting a "flaky" file read up to 3 times before giving up and logging failure.

Milestone 2 — Data Structures, Functions & Basic OOP, String Manipulation

Q2.1 — Refactor `compute_cost`'s rate table into a module-level `RATES` dict; rewrite the function to use `.get(call_type, RATES["domestic"])`.

Q2.2 — Define a `Subscriber` class (`msisdn`, `plan_type`, a `calls` list) with a method `total_cost(self) -> float`.

Q2.3 — Define a `CDR` class with a method `is_suspicious(self) -> bool` implementing the two fraud rules from FR5.

Q2.4 — Using a `set`, compute the unique `msisdn` values in the CDR batch that do NOT exist in the subscriber master list (FR7).

Q2.5 — *(String manipulation)* Some legacy CDR sources deliver a pipe-delimited string, e.g. `"9876543210|domestic|180"`. Write `parse_legacy_line(line: str) -> dict` using `.split("|")` and `.strip()`.

Q2.6 TRICKY — Why is a `set` the right choice for "unknown subscribers" rather than a `list`? What goes wrong with a list if the same unknown `msisdn` appears in 50 CDRs?

Milestone 3 — File Handling (CSV/JSON) & Scripting Best Practices

Q3.1 — Write `load_subscribers(json_path: str) -> dict` keyed by `msisdn`, valued by `Subscriber` instances.

Q3.2 — Write `load_cdrs(csv_path: str) -> list` using `csv.DictReader`, applying `parse_cdr_line` inside a `try/except ValueError`, logging and skipping malformed rows.

Q3.3 — Write `write_report(report: dict, output_path: str) -> None` writing indented JSON.

Q3.4 — Wire everything together in `main()` using `argparse` (`--subscribers`, `--cdrs`, `--output`, `--fraud-threshold-sec` default 3600). Configure `logging` once, at the start.

Q3.5 — Implement NFR2: if >10% of CDR rows were malformed, log `CRITICAL` and `sys.exit(1)` instead of writing a report.

Q3.6 TRICKY — A teammate suggests a broad `except Exception:` around `load_cdrs` "just to be safe." Why is catching `ValueError` specifically better? What would broad-except hide if the CSV file didn't exist at all?

Project 2 — Retail Inventory & Sales Reconciliation CLI

> **Client:** Northfield Retail Co. (fictitious mid-size retail chain) · **Requested by:** Finance & Store Operations, via the Engineering Lead

2.1 Problem Statement / PRD

Background. Each store emails a daily sales export (CSV) to HQ. Finance currently reconciles this by hand against the inventory system's snapshot (JSON) to catch stock issues and compute revenue — a slow, manual, spreadsheet-driven process. Engineering has scoped a CLI tool to automate daily reconciliation for a single store's batch. You are the engineer assigned to build it.

Goals

- Automate ingestion and validation of daily sales + inventory data.
- Compute revenue per transaction, correctly applying discounts.
- Flag stock issues: oversold SKUs and low-stock SKUs.
- Produce a daily reconciliation report, machine-readable (JSON) and human-readable (console).

Out of Scope (Phase 2, not this project)

- No live POS/database integration — file-based batch processing only.
- No multi-day trend analysis or forecasting.
- No user interface.

Stakeholders

- **Finance Analyst** — consumes the daily report for revenue reconciliation.
- **Store Ops Manager** — needs the stock-issue flags to act on same-day.

Functional Requirements

ID	Requirement
FR1	The tool shall load a product inventory snapshot from a JSON file (`sku` , `name` , `unit_price` , `stock_qty` , `category`).
FR2	The tool shall load a batch of sales transactions from a CSV file (`sku` , `qty_sold` , `discount_pct` , `channel`).

FR3	The tool shall validate each sales row and skip (logging + counting) malformed rows without crashing the batch.
FR4	The tool shall compute line revenue as `qty_sold * unit_price * (1 - discount_pct/100)`, rounded to 2 decimal places.
FR5	The tool shall flag a SKU as **oversold** if total `qty_sold` across the batch exceeds its `stock_qty`.
FR6	The tool shall flag a SKU as **low-stock** if remaining stock falls below a configurable reorder threshold (default 10), tracked with a set.
FR7	The tool shall identify sales rows referencing a `sku` not present in inventory, reported separately as "unknown SKU" anomalies.
FR8	The tool shall aggregate total revenue by `category` and by `channel` (`online` / `in-store`).
FR9	The tool shall write a daily report as indented JSON, and print a human-readable console summary.
FR10	Input file paths and the low-stock reorder threshold shall be configurable via CLI arguments.

Non-Functional Requirements

ID	Requirement
NFR1	All status/error events must be logged with timestamps and severity levels — no bare `print` for this.
NFR2	If more than 10% of sales rows in a batch are malformed, the tool must log `CRITICAL` and exit with a non-zero exit code instead of writing a partial report.
NFR3	Code must be organized into separate modules (parsing/calculations/reporting/IO), not one script.
NFR4	All monetary values must be rounded to 2 decimal places.

Acceptance criteria. Running the tool against the provided sample `inventory.json` and `sales.csv` produces a `report.json` whose revenue totals and stock flags you can verify by hand-calculating at least two SKUs yourself.

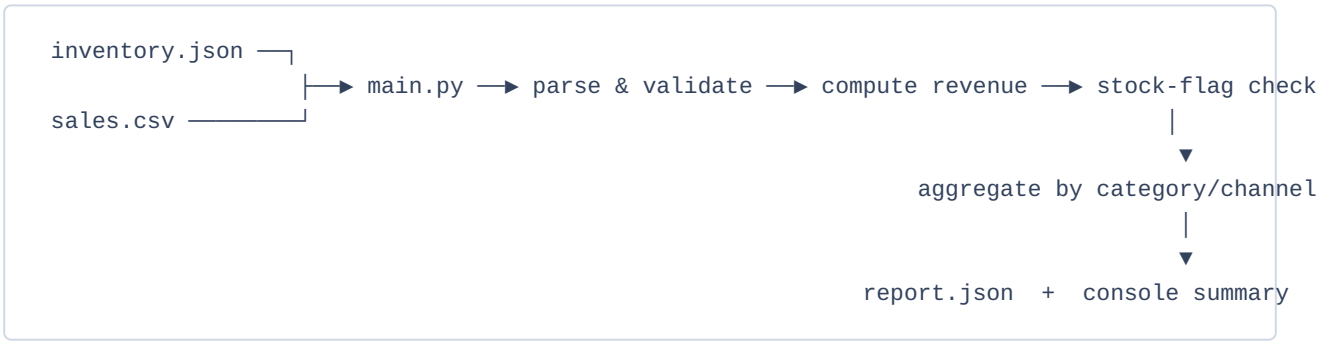
2.2 Architecture (Simple)

Module layout:

```
retail_cli/
    main.py          # CLI entry (argparse), orchestrates the pipeline
    config.py          # default reorder threshold -- the ONLY magic number
    io_utils.py           # load_inventory(), load_sales(), write_report()
    models.py                # Product class, SaleTransaction class
```

```
    calculations.py           # compute_revenue(), stock flag rules
    reporting.py                 # build_report()
```

Data flow:



Layered view:

Layer	Responsibility	Modules
CLI Layer	Parse arguments, wire everything together, set exit code	`main.py`
Business Logic	Revenue calculation, stock-flag rules, aggregation	`calculations.py`, `reporting.py`
Data Access	Reading/writing files	`io_utils.py`

2.3 Sample Data

Provided alongside this document in `python_core_capstone_workbook.ipynb` — running the sample-data cell generates `inventory.json` and `sales.csv`, including an oversold SKU, a low-stock SKU, an unknown SKU, and a malformed row.

Milestone 1 — Setup, Syntax, Variables & Data Types, Control Flow & Loops

> **Q1.1** — Write a startup banner print using an f-string including today's date, guarded to run only when executed directly.

> **Q1.2** — Write `parse_sale_row(row: dict) -> dict` casting `qty_sold` to `int` and `discount_pct` to `float`, raising `ValueError` on bad/missing data.

> **Q1.3** — Write `compute_revenue(unit_price, qty_sold, discount_pct) -> float`. Using `if` control flow, clamp `discount_pct` to 0–100 if out of range.

Q1.4 — Using a `for` loop, iterate over a hardcoded list of at least 4 sample sale dicts and print each computed revenue.

Q1.5 TRICKY — Using `while`, simulate a retry loop for a "flaky" file read, up to 3 attempts.

Milestone 2 — Data Structures, Functions & Basic OOP, String Manipulation

Q2.1 — Build `revenue_by_category` and `revenue_by_channel` dicts, accumulated while processing sales (FR8).

Q2.2 — Define a `Product` class (`sku`, `name`, `unit_price`, `stock_qty`, `category`) with a method `remaining_stock(self, qty_sold: int) -> int`.

Q2.3 — Define a `SaleTransaction` class with a method `line_revenue(self, product) -> float`.

Q2.4 — Using a `set`, compute unknown SKUs sold (FR7) and low-stock SKUs after this batch (FR6).

Q2.5 — *(String manipulation)* SKUs follow `"SKU-<category_code><number>"`. Write `category_code_from_sku(sku: str) -> str` using string slicing, as a cross-check against the declared category.

Q2.6 TRICKY — Why is a `set` the right choice for tracking low-stock SKUs across a batch with repeated rows for the same SKU, rather than a `list`?

Milestone 3 — File Handling (CSV/JSON) & Scripting Best Practices

Q3.1 — Write `load_inventory(json_path: str) -> dict` keyed by `sku`, valued by `Product` instances.

Q3.2 — Write `load_sales(csv_path: str) -> list` using `csv.DictReader`, applying `parse_sale_row` inside a `try/except ValueError`, logging and skipping malformed rows.

Q3.3 — Write `write_report(report: dict, output_path: str) -> None` writing indented JSON.

Q3.4 — Wire everything together in `main()` using `argparse` (`--inventory`, `--sales`, `--output`, `--reorder-threshold` default 10). Configure `logging` once, at the start.

Q3.5 — Implement NFR2: if >10% of sales rows were malformed, log `CRITICAL` and `sys.exit(1)` instead of writing a report.

> **Q3.6** TRICKY — Why is catching `ValueError` specifically in `load_sales` better than a broad `except Exception:`? What would broad-except hide if `sales.csv` had the wrong file encoding entirely, versus just one bad row?

Deliverables Checklist & Evaluation Rubric

Applies to both projects.

Deliverables Checklist

- ☐ Package structure matches the Architecture section — no single monolithic script.
- ☐ A `README.md` with setup and run instructions.
- ☐ Running `main.py` end-to-end against the provided sample data produces a correct `report.json`.
- ☐ A human-readable console summary prints on run.
- ☐ `logging` used throughout for status/errors — no `print` outside the final summary.
- ☐ Malformed-row handling demonstrated — the batch doesn't crash on bad rows.
- ☐ Exit-code behavior implemented (NFR2) — verified by deliberately feeding a batch with >10% bad rows.
- ☐ Code follows the shared Coding Standards section (docstrings, naming, no bare except, no magic numbers).

Evaluation Rubric

Criteria	Weight	What "Good" Looks Like
Correctness	40%	Report totals match hand-calculated expected values on the sample data.
Code Quality & Standards	25%	PEP8-clean, docstrings present, naming conventions followed, modular structure matching the architecture.
Error Handling & Robustness	20%	Malformed rows logged and skipped, correct exit-code behavior, no bare/broad except.
Documentation	15%	A README clear enough that a teammate could run the tool without asking you questions.

> **Closing note.** Both projects deliberately reuse the same shape: load → validate → compute → flag → aggregate → report. That's not an accident — it's the same shape you'll see again, at larger scale, in

> the FastAPI capstone project later in the curriculum. Getting comfortable with this shape now, using nothing but core Python, pays off directly later.